



BLUE BITS

البرمجة التفرعية

نظري (5)

ملف داعم

2026/2025

السنة الرابعة



مكتبة شهاب
الشهاب الذبيبة - يفرق بيننا وبين الناس

[f](#) [@](#) [v](#) [t](#) /BlueBitsTeam

www.bluebitsteam.com

MPI: Non – blocking Routines

تقوم الروتينات غير الحجزية بإرجاع العملية فوراً.

- وبذلك تمكّن البرنامج من تنفيذ الأمر التالي بغض النظر عما إذا كان الروتين قد اكتمل كلياً أم لا
- الإرسال غير الحجز *MPI – Isend()* : حيث يتم إرجاع العملية فوراً حتى قبل أن يتم تأمين مكان المصدر
- الاستقبال غير الحجز *MPI – Irecv()* : حيث يتم إرجاع العملية فوراً دون التأكد من استقبال الرسالة
- صيغ *Non – blocking Routines* هي:

MPI – Isend (buf, count, data type, dest, tag, comm, request)

MPI – Irecv (buf, count, data type, source, tag, comm, request)

request : يتم من خلالها التأكد إذا أرسل أو استقبل الطرف الثاني أم لا

يتم تحديد انتهاء العمليات باستخدام روتينات منفصلة

- MPI – Test()* : يقوم بإرجاع العلامة (*flag*) على الفور مشيراً إلى ما إذا كانت العملية قد اكتملت في ذلك الوقت أم لا

- MPI – wait()* : يقوم بالانتظار حتى يتم الانتهاء من العملية ثم يعيد البرنامج.

تحتاج هذه الروتينات إلى ربطها بعملية معينة، ويتم ذلك باستخدام نفس المعامل (*request*) في كليهما

Message Completion and Buffering

سؤال دورة

```
* buf = 3;
MPI – Send (buf, 1, MPI – INT,...);
* buf = 4; /* ok, receiver will always receive 3 */
```

هنا قيمة *buf = 3* وتم عملية الإرسال المتزامنة وذلك بال *Send* يصل إلى الطرف الآخر قيمة *buf = 3* ولا يقوم بتعديل قيمة *buf* حتى يضمن أن الطرف الآخر قام بالاستلام لأن *send* من نوع *blocking* (متزامنة) أي يتم التأكد من صحة الإرسال والاستقبال وقادرة على تحديد ما هي القيمة التي تم استلامها بالجهة الأخرى

```
* buf = 3;
MPI – Isend (buf, 1, MPI – INT,...);
* buf = 4; /* undefined whether the receiver will get 3 or 4 */
```

هنا سيستقبل الطرف الثاني قيمة *3 or 4* لأن *isend* من نوع (*non – blocking* غير متزامنة) أي لا يتم التأكد من صحة الاستلام للرسالة وغير قادرة على معرفة القيمة التي تم إرسالها هي *3 أو 4*

MPI – Waitsome

يتم تحديد مجموعة من العناصر مخزنة بمصفوفة للتأكد من استقبال بعض من هذه العناصر أي انتظار بعض من العمليات.

*MPI – Waitsome (int incount, MPI – Request array – of – requests[], int * outcount, int .. array_of_indices[], MPI – status array – of – Statuses[])*



- بارامترات الدخل:
int incount: كم عملية لازم أنتظر حتى تنتهي (طول مصفوفة ال *request*)
array_of_requests: تخزن جميع ال *requests* القادمة من *isend* وتفحص إذا كان عددها يساوي عدد *incount* أي تم تنفيذ العملية ويتم إكمال باقي العمل.
- بارامترات الخرج:
int outcount: عدد الطلبات المكتملة (كم معالجة انتهى تنفيذها)
array - of - indices: تُعطي *id* المعالجات التي اكتملت مهمتها
array_of_statuses: وهي مصفوفة مؤشرة لحالة العمليات التي اكتملت (من أي عملية استقبلت حجم البيانات , التاغ , المعالجة المصدر) .
أنتظر بعض العمليات غير المتزامنة على أنها اكتملت
- مثلاً لدينا حلقة *for* بداخلها تعليمة *Isend* تنفيذها مكرر
إذا عملت *wait* لازم أتأكد بكل مرة المعالجة خلصت أم لا
أما *wait some* يعطيه رقم معين لمجرد وصوله إلى هذا الرقم تم تنفيذ العملية

MPI_Waitany

يكفي أن تكتمل مهمة *Process* واحدة لتطلع (أي انتظار أي من الطلبات ولا تكون محدّدة) أي أن أي عملية تم استقبالها وتم التأكد أنها وصلت بشكل صحيح سليم فتكفي لاستكمالها باقي العمليات بين المجال *0 to count-1* (أي وحدة لا على التعيين ضمن المصفوفة التي أقوم بمراقبتها)
*MPI - Waitany(int count, MPI - Request array - of - requests [], int * index, MPI - Status * status)*

- بارامترات الدخل:
count: طول اللائحة
Array - of - requests: مصفوفة الطلبات

- بارامترات الخرج:
Index: هو ال *Index* الخاص بالعملية التي اكتملت ضمن المجال من *0* إلى *count - 1*
status: حالة ال *object*

MPI_Waitall

انتظار انتهاء عمل جميع ال *Processes* ثم الخروج أي ننتظر كل الطلبات حتى تكتمل وتصل بشكل سليم

MPI - Waitall(int count, MPI - Request array - of - requests [], MPI - Status array - of - statuses [])

- بارامترات الدخل:
count: طول اللائحة
array_of_requests
- بارامترات الخرج:
array_of_statuses



Test function

*int MPI_Test(MPI_Request * request, int * flag, MPI_Status * status)*

عملية اختبار *request* معينة، الهدف منها هو التحقق من اكتمال عملية الإرسال والاستقبال يعيد التابع قيمة *flag* مؤشر وهو يقوم باختيار *request* معينة إذا $flag = 1 \Rightarrow true$ أي العملية اكتملت إذا كان $flag = 0 \Rightarrow false$ لم تكتمل العملية إذا كان الطلب يشير إلى عملية *receive* وتم إرجاع $Flag = 1$ فإن حالة البارامتر تحتوي على معلومات عن الرسالة المستلمة على أنها مصنفة لـ *MPI - Recv()* تكون حالة البارامتر غير محددة إذا لم تكتمل بعد عملية الاستلام المحددة. إذا كان الطلب يشير إلى عملية إرسال جميع إدخالات الـ *status* ما عدا *status.MPI_ERROR*. تكون غير معرّفة *undefined*

Example: Non blocking operations

قسم التصريحات

```
1 MPI_Request reqs[4];
2 MPI_Status stats[4];

3 buf[0] = -1;
4 buf[1] = -1;
5 MPI_Init(&argc, &argv);
6 MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
7 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
8 prev = rank - 1;
9 next = rank + 1;
```

يتم التصريح عن *reqs* و *stats* كـ *global* دائماً

```
10 if (rank == 0) prev = numtasks - 1;
11 if (rank == (numtasks - 1)) next = 0;

12 MPI_Irecv(&buf[0], 1, MPI_INT, Prev, tag1, MPI_COMM_WORLD, &reqs[0]);
13 MPI_Irecv(&buf[1], 1, MPI_INT, next, tag2, MPI_COMM_WORLD, &reqs[1]);
MPI_Isend(&rank, 1, MPI_INT, Prev, tag2, MPI_COMM_WORLD, &reqs[2])
```

إذا كانت العملية 0 نجعل العملية التي قبلها هي آخر عملية في الـ *communicator*
إذا كانت العملية هي الأخيرة نجعل العملية التي بعدها هي العملية 0

- تستقبل من العملية التي قبلها
- تستقبل من العملية التي بعدها
- ترسل إلى العملية التي قبلها



- ترسل إلى العملية التي بعدها

```
14 MPI - Isend(&rank, 1, MPI - INT, next, tag1, MPI - COMM - WORLD, &reqs[3]);  
15 printf("BeforeWaitall: I am %d and recieved buf[0] = %d, buf[1] = %d \\  
n", rank, buf[0], buf[1]);  
16 MPI_Waitall(4, reqs, stats);  
17 printf("AfterWaitall: I am %d and recieved buf[0] = %d, buf[1] = %d \\  
n", rank, buf[0], buf[1]);  
18 MPI_Finalize(); }
```

1: عدد العناصر المُرسلة أو المُستقبلة

Prev: تشير إلى العملية السابقة

next: تشير إلى العملية اللاحقة

Request: لازم تتميز عن بعضها البعض

tag: يجب أن يكون الوسم مشترك بين العملية المُرسلة والمُستقبلة

حسب المثال السابق: العملية 1 تُرسل قيمة واحدة إلى العملية 2 ، نلاحظ أن كليهما يمتلكان نفس الوسم وهو *tag2*

Deadlock

يتم فيه حظر مجموعة من العمليات لأن كل عملية تحتفظ بمورد وتنتظر مورداً آخر تحصل عليه من عملية أخرى (أي عمليتان تنتظران بعضهما والمزاحمة تحدث على المصدر المشترك)

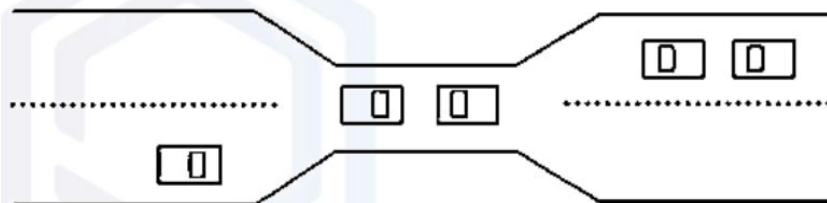
1. **مثال:** إذا كان لدينا قطاران يقتربان من بعضهما على نفس المسار ولا يوجد سوى مسار واحد ولا

يمكن لأي قطار أن يتحرك عندما يصبحان يواجهان بعضهما البعض

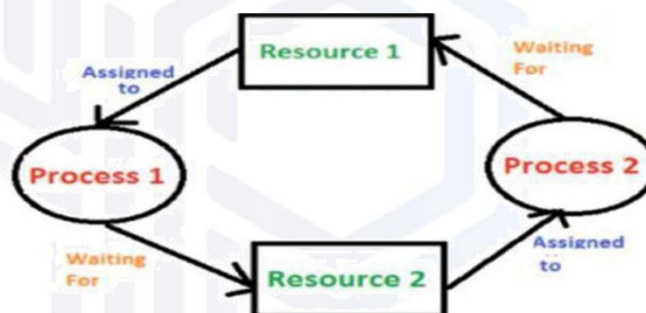
2. **مثال:** عشاء الفلاسفة (*Google it*)

3. **مثال:**

Bridge Example:



4. **مثال:** موقف مماثل في نظام التشغيل عندما يكون هناك عمليتان أو أكثر تحتفظ ببعض الموارد وتنتظر الموارد التي تحتفظ بها عمليات أخرى





في الرسمة نلاحظ العملية الأولى تحتفظ بالموارد 1 وتنتظر المورد 2 الذي تم الحصول عليه من العملية 2 والعملية 2 تنتظر المورد 1

أنواع الموارد: $R1, R2, \dots, RM$ يمكن أن تكون *cycles* من *cpu* و *I/o* لمساحة التخزين تستخدم العملية في أنظمة التشغيل موارد مختلفة وتستخدم بطريقة:

1. *Requests a resource* تطلب مورد

2. *Use the resource* تستخدم مورد

3. *Releases the resource* تطلق مورد

كيف يتم حدوث *deadlock* تفرعياً؟ تتزامن العمليات على ال *buffers* او الذاكرة او المعالج

DeadLock Characterization

يمكن أن يحدث *deadlock* إذا تحققت الشروط الأربعة بشكل متزامن

1. *Mutual Exclusion* (الاستبعاد المتبادل): مورد أو أكثر غير قابل للمشاركة (يستخدم المورد الواحد من قبل عملية واحدة فقط في المرة الواحدة)
2. *Hold and wait*: تحتفظ العملية بمورد واحد على الأقل وتنتظر موارد أخرى
3. *No Preemption*: لا يمكن أخذ المورد من عملية إذا لم تقم هذه العملية من تركه
4. *Circular Wait*: مجموعة من العمليات تنتظر بعضها بشكل دائري

Deadlocks with Point-to-Point Communications

Send and receive operations must be used with care.

/* program fragment which always causes a deadlock */

MPI Comm rank (comm, &my rank);

if (my rank == 0) {

MPI Recv (recvbuf, count, MPI INT, 1, tag, comm, &status);

MPI Send (sendbuf, count, MPI INT, 1, tag, comm);

}

else if (my rank == 1) {

MPI Recv (recvbuf, count, MPI INT, 0, tag, comm, &status);

MPI Send (sendbuf, count, MPI INT, 0, tag, comm);

}

لأن كلا العمليتين 0 و 1 تنفذ العملية *MPI - recv()* قبل عملية *MPI - Send()* لدينا حالة *deadlock*

وهذا يؤدي إلى طريق مسدود بسبب الانتظار المتبادل

بالنسبة للعملية 0 لا يمكن بدء تشغيل *MPI - Send()* قبل اكتمال عملية *MPI - recv()* السابقة

يكون هذا ممكناً عندما تُنفَّذ العملية (1) *MPI - Send()* الخاصة بها ولكن لا يمكن أن يحدث هذا لأن

العملية (1) يجب أن تكمل عملية *MPI - recv()* السابقة أولاً

البرنامج يؤدي دائماً إلى *Deadlock*



ملاحظة:

في العمليات من نوع *blocking* لتجنب حالات *deadlock* يجب على العمليات أن تُرسل قبل أن تستقبل دائماً (p2p)

ملاحظة:

يمكن تجنب حالات *deadlock* في المثال السابق باستخدام إحدى الطرق:

- (1) استخدام *buffer* خارجي خاص بنظام التشغيل يدعى *System buffer*
- (2) تعيين لكل *process* وحدة تخزين *buffer* خاص بها
- (3) تحديد أي العمليات التي سترسل وأي منها ستستقبل باستخدام *Order of Routines*
- (4) استخدام *Waitall*, *Waitany*, *Waitsome*, إذا كانت العمليات من نوع *non – blocking*

Deadlocks with Point-to-Point Communications

/* program fragment for which the occurrence of a deadlock depends on the implementation */

MPI Comm rank (comm, &my rank);

if (my rank == 0) {

MPI Send (sendbuf, count, MPI INT, 1, tag, comm);

MPI Recv (recvbuf, count, MPI INT, 1, tag, comm, &status);

}

else if (my rank == 1) {

MPI Send (sendbuf, count, MPI INT, 0, tag, comm);

MPI Recv (recvbuf, count, MPI INT, 0, tag, comm, &status);

}.

The occurrence of a deadlock might also depend on whether the runtime system uses internal system buffers or not.

حالة *deadlock* لأن العملية الأولى عم تعمل *send* لل *process* رقم 1 والثانية عم تعمل *send* لل *process* رقم 0 ولا يوجد أحد يستقبل لكي تكتمل عملية ال *send* قد يعتمد حدوث ال *deadlock* على إذا كان نظام التشغيل يستخدم مخازن داخلية (*internal system buffers*) أو لا

يمكن ألا يحدث *deadlock* في الحالة السابقة ويتم إرسال الرسائل بشكل صحيح دون توقف إذا كان نظام التشغيل *mpi* يستخدم *system buffers* وفي هذه الحالة تم نسخ الرسالة المرسله بواسطة العملية 1 و 0 أولاً من مخزن الإرسال المؤقت المحدد *sendBuf* إلى مخزن مؤقت للنظام *system buffer* قبل الإرسال الفعلي وبالتالي بعد عملية النسخ هذه تكتمل عملية الإرسال *MPI_Send()* لأنه يمكن إعادة استخدام مخازن الإرسال المؤقتة

وبالتالي يمكن لكلتا العمليتين 0 و 1 تنفيذ عملية *MPI – Recv()* الخاصة بهما ولن يحدث *deadlock*



- يحدث قفل *deadlock* إذا كان نظام التشغيل لا يستخدم مخازن النظام *system buffer* أو كانت مخازن النظام المستخدمة صغيرة جداً
- في هذه الحالة لا يمكن لأي من العمليتين إكمال عملية *MPI - Send()* بما أن عملية الاستقبال لن تُنفَّذ بواسطة العملية الأخرى

Deadlocks with Point-to-Point Communications

```
/* program fragment that does not cause a deadlock */
MPI Comm rank (comm, &myrank);
if (my rank == 0) {
  MPI Send (sendbuf, count, MPI INT, 1, tag, comm);
  MPI Recv (recvbuf, count, MPI INT, 1, tag, comm, &status);
}
else if (my rank == 1) {
  MPI Recv (recvbuf, count, MPI INT, 0, tag, comm, &status);
  MPI Send (sendbuf, count, MPI INT, 0, tag, comm);
}
```

The **secure Implementation** which does not cause deadlocks even if **no system buffers** are used.

هنا لا يوجد حالة *deadlock* التنفيذ الآمن لا يسبب حالة *deadlock* حتى لو لم يتم استخدام مخازن مؤقتة للنظام

secure program

البرنامج آمن أي صحة نتائج هذا البرنامج لا تعتمد على واصفات محددة من *MPI runtime system* مثل وجود *system buffers* أو حجم هذه الـ *buffers*.

يعمل *Secure MPI program* بشكل صحيح حتى ولو لم نستخدم *System buffers* على الإطلاق إذا قامت أكثر من عمليتين بتبادل الرسائل بحيث تقوم كل عملية بإرسال واستقبال رسالة فيجب أن يحدد البرنامج بالضبط الترتيب الذي سيتم به تنفيذ عمليات الإرسال والاستقبال لتجنب حالة الـ *deadlock*.

مثال:

نعتبر برنامج يحتوي على عدد p من العمليات حيث العملية i ترسل رسالة إلى العملية $((i + 1) \bmod p)$ وتستقبل رسالة من العملية $(i + p - 1) \bmod p$ من أجل $0 \leq i \leq p - 1$ وبالتالي يتم إرسال الرسالة بحلقة منطقية.

Secure Implementation

تضمن التناوب بين عمليات الإرسال والاستقبال

- يمكن الحصول عليه إذا قامت العمليات ذات الرتبة الزوجية بتنفيذ عملية الإرسال أولاً ثم عملية الاستقبال.
- بينما تقوم العمليات ذات الرتبة الفردية بتنفيذ عملية الاستقبال أولاً ثم عملية الإرسال.



ملاحظة:

إذا كان عدد العمليات الفردية = عدد العمليات الزوجية، التناوب يكون صحيح ولن نحتاج لاستخدام الـ *wait* ، أمّا إذا كان عدد العمليات الفردية $\neq$ عدد العمليات الزوجية يجب أن نستخدم هنا الـ *wait* مع التذكير أنّ العمليات تعمل على التفرّع.

تذكير:

phase 1 يتم تنفيذها من قبل العمليات الـ 4 بنفس الوقت

في حالة تساوي عدد العمليات

Phase	Process 0	Process 1	Process 2	Process 3
1	MPI Send() to 1	MPI Recv() from 0	MPI Send() to 3	MPI Recv() from 2
2	MPI Recv() from 3	MPI Send() to 2	MPI Recv() from 1	MPI Send() to 0

في حالة عدم تساوي عدد العمليات

Phase	Process 0	Process 1	Process 2
1	MPI Send() to 1	MPI Recv() from 0	MPI Send() to 0
2	MPI Recv() from 2	MPI Send() to 2	-wait-
3		-wait-	MPI Recv() from 1

Sendrecv Operation

هذه العملية حظرية وتجمع بين عملية إرسال واستقبال في استدعاء واحد المعلومات لها المعاني التالية:

- يحدد *sendbuf* مخزن الإرسال الذي تُخزن فيه عناصر البيانات المراد إرسالها
- هو عدد العناصر المراد إرسالها *sendcount*
- هو نوع بيانات العناصر في مخزن الإرسال *sendtype*
- هو رتبة العملية المستهدفة التي تُرسل إليها عناصر البيانات *dest*
- هو علامة الرسالة المراد إرسالها *sendtag*
- هو مخزن الاستقبال للرسالة المراد استقبالها *recvbuf*
- هو الحد الأقصى لعدد العناصر المراد استقبالها *recvcount*
- هو نوع بيانات العناصر المراد استلامها *recvtype*
- هو رتبة العملية التي يُتوقع أن تأتي منها الرسالة *source*
- هو الوسم المتوقع للرسالة المراد استلامها *recvtag*
- هو أداة الاتصال المستخدمة في عملية الاتصال *comm*
- يحدد بنية البيانات لتخزين معلومات الرسالة *status*

```
int MPISendrecv(void*sendbuf, int sendcount, MPIDatatype sendtype, int dest, int sendtag,
void*recvbuf, int recvcount, MPIDatatype recvtype, int source, int recvtag, MPIComm comm,
MPIStatus*status)
```



يمكن اعتبار انه تابع واحد يضمن أنه لا يوجد *deadlock* وتم دمج البارامترات الموجودة في *send* و *receive* باستخدام *MPI - Sendrecv()* لا يحتاج المبرمج إلى القلق بشأن ترتيب عملية الإرسال والاستلام البارامترات *sendBuf* و *recvBuf* تحدد المخازن المؤقتة للإرسال والاستلام للعملية المنفذة يجب أن تكون البارامترات منفصلة وموقع الذاكرة غير متداخل والمخازن المؤقتة قد يكون لها أطوال مختلفة، والمدخلات المخزنة ممكن أن تكون عناصرها مختلفة في نوع البيانات *data types*

MPI_Sendrecv_replace Operation

```
int MPI Sendrecv replace (void *buffer, int count, MPI Datatype type, int
dest, int sendtag, int source, int recvtag, MPI Comm comm, MPI Status
*status)
```

If The send and receive buffer are **identical**, this operation is also blocking

Here, buffer specifies the buffer that is used as both send and receive buffer. For this function, count is the number of elements to be sent and to be received; these elements now should have identical type type.

يحدد ال *buffer* باعتباره *buffer* مشترك لعمليتي الإرسال والاستقبال معا من أجل هذا التابع: المتغير *count* يمثل عدد العناصر المرسل والمستقبل؛ ويجب ان تكون هذه العناصر من نفس النوع

Deadlocks with Collective Communication

We consider two MPI processes which execute two MPI Bcast() operations in opposite order

```
switch (my rank) {
case 0: MPI Bcast (buf1, count, type, 0, comm);
        MPI Bcast (buf2, count, type, 1, comm);
        break;
case 1: MPI Bcast (buf2, count, type, 1, comm);
        MPI Bcast (buf1, count, type, 0, comm);
}
```

Source????

تابع يستخدم إذا كان الإرسال والاستقبال متطابقان وهذه العملية من نوع *blocking* نضع متحول واحد وهو *buffer* فقط إذا كان هو المرسل يكون ممتلئ بينما إذا كان المستقبل يكون فارغ وبعد التنفيذ يصبح ممتلئ بالقيام بتنفيذ البرنامج السابق سوف يؤدي إلى حالتين خطأ:



1. قد يطابق *MPI runtime system* مع أول استدعاء *(MPI - Bcast)* لكل عملية، القيام بهذا يؤدي إلى حدوث خطأ لأن العمليتين تحددان *roots* مختلفة
2. قد يطابق *runtime system* مع استدعاء *(MPI - Bcast)* مع نفس الـ *root* مثلما تم وضعه من قبل المبرمج

الـ *deadlock* يحدث إذا كانت الـ *buffers* صغيرة أو لم يتم استخدام *system buffers*

عمليات الاتصال من نمط *collective* دائماً تكون من نوع *blocking*، لذلك تعتبر عمليات متزامنة إذا لم يتوفر *buffers* في النظام (أو كانت صغيرة)

لذا فإن أول استدعاء لـ *(MPI - Bcast)* سيجعل الـ *process 0* في حالة *block* إلى أن تقوم الـ *process 1* باستدعاء تابع *Bcast* خاصتها والذي يحمل نفس الجذر *root*

لكن ذلك لن يحدث لأن الـ *process 1* بعد ذاتها ضمن حالة *block* بسبب أول استدعاء *MPI - Bcast* لها حيث أنها تنتظر الـ *process 0* أن تستدعي الـ *Bcast* الثاني خاصتها

وهذا الوضع الكلاسيكي لحالات الـ *deadlock* التي درسناها مسبقاً

يمكننا تفادي هذه الحالة إذا قامت كل الـ *processes* باستدعاء توابعها الخاصة متطابقة وب نفس الترتيب مع الـ *processes* الأخرى

يحدث لدينا حالة *deadlock* أخرى عند دمج النمط *collective* مع أسلوب *single transfer* وهو موضح بالكود التالي:

```
switch (my rank) {
    case 0: MPI Bcast (buf1, count, type, 0, comm);
            MPI Send (buf2, count, type, 1, tag, comm);
            break;
    case 1: MPI Recv (buf2, count, type, 0, tag, comm, &status);
            MPI Bcast (buf1, count, type, 0, comm);
}
```

في هذا الكود قد تدخل الـ *processes* بحالة *deadlock* مشابهة للسابقة إذا لم يكن هناك *buffers* ضمن البرنامج الأساسي

Example: Deadlocks with Collective Communication

مثال شامل:

```
int my_rank, size;
int recvbuf;
int count = 1, tag1 = 100, tag2 = 101;
MPI_Status status;
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
if (my_rank == 0) {
    MPI_Recv(&recvbuf, count, MPI_INT, 1, tag1, MPI_COMM_WORLD, &status);
```



```
MPI_Send(&my_rank, count, MPI_INT, 1, tag2, MPI_COMM_WORLD); }  
    else if (my_rank == 1) {  
MPI_Recv(&recvbuf, count, MPI_INT, 0, tag2, MPI_COMM_WORLD, &status);  
MPI_Send(&my_rank, count, MPI_INT, 0, tag1, MPI_COMM_WORLD);  
}
```


2026/2025

السنة الرابعة



BLUE BITS



أعضاء الفريق



الفريق التقني



الفريق العلمي

إبراهيم هبرة	عائشة العُمر العلّوش	إيناس بيلوني	ظلال بابا
إسراء حاج حمدي	عمر قظان	تسنيم طالب	عائشة قلّه
بتول لبّان	فاطمة الحسين	حلا إبراهيم	عائشة مُصطفى
بشرى كوكش	قاسم يحيى	دلال تسقيه	قمر كمد
بنيامين حكمان	كريست ياغجيان	ديانا صبّاغ	محمد يمان جلال
تسنيم حوكان	محمد منير	زهير حّقامي	محمود غزال
تسنيم نور الدّين	مديحة جّمّال	شكرية سُلطان	وفاء حّقامي
جميل هلال	ميرا فضلي	شهد كرّوم	يوسف نيّال
جودي جلال	نجم الدّين درويش		
خديجة بيازيد	هبة الله كتلو		
رشا معراوي	وداد قلعيّة		
سدرة مبيض	ياسر صبّاغ		

هل لديك أي ملاحظة؟ لا تتردّد في مراسلتنا.

